

POSE — Object-Oriented Programming in C#

2AHIF, School Year 2026/27

HTL Leonding

2026-09-30

Table of contents

Introduction	3
1 OO Basics	4
1.1 From Structured to Object-Oriented	6
1.2 Classes and Objects	7
1.3 Constructors	11
1.4 Properties in Detail	14
1.5 Static vs. Instance	19
1.6 Enumerations (<code>enum</code>)	22
1.7 Naming Conventions	24
1.8 UML Class and Object Diagrams	26
1.9 Glossary	31

Introduction

This book collects the course material of **POSE — Object-Oriented Programming in C#** for class 2AHIF at HTL Leonding, school year 2026/27. It grows over the year, one part per unit. Each part holds the theory of its unit and the live demos from class.

The exercises are not part of the book: you get them through Classroom50.

Tools you need

- The [.NET 10 SDK](#)
- One of these IDEs:
 - [JetBrains Rider](#)
 - [Visual Studio](#)
 - [Visual Studio Code](#) with the [C# Dev Kit](#) extension
- [PlantUML](#), to view the UML diagrams in the Readmes and exercises and to draw your own. In this book the diagrams are already drawn for you, but on GitHub and in your IDE a diagram is only text until PlantUML renders it:
 - **Rider:** open *Settings* → *Languages & Frameworks* → *Markdown* and enable **PlantUML** under *Markdown Extensions*. The Markdown preview then shows the diagrams.
 - **Visual Studio Code:** install the [PlantUML](#) extension. The Markdown preview (**Ctrl+Shift+V**) then shows the diagrams. The extension draws them on your computer, which needs [Java](#) 11 or newer. Without Java, set `plantuml.render` to `PlantUMLServer` in the settings.
 - **Visual Studio:** its Markdown preview can't show PlantUML diagrams. Paste the diagram text into the [PlantUML online server](#), which needs no installation, or open the file in VS Code.
- [Git](#) and a [GitHub](#) account

Part 1

00 Basics

i Why object orientation?

Object-oriented programming is the foundation most modern software is built on, because organizing code around objects that bundle data and behavior together is what makes large, evolving systems stay manageable.

This unit introduces classes and objects in *C#*, using a *Pupil* class as the running example. The theory is split into these chapters:

1. [From Structured to Object-Oriented](#): what objects are and why we use them
2. [Classes and Objects](#): defining a class, fields, and creating objects with `new`
3. [Constructors](#): bringing a new object into a working state
4. [Properties in Detail](#): auto, computed and fully written-out properties
5. [Static vs. Instance](#): members shared by all objects of a class, and `const` vs. `readonly` vs. `static`
6. [Enumerations](#): named constant values with `enum`
7. [Naming Conventions](#): how to name classes, properties, fields and methods
8. [UML Class and Object Diagrams](#): the PlantUML notation used in the exercises, and snapshots of the objects at runtime
9. [Glossary](#): the most important terms of this unit

1.1 From Structured to Object-Oriented

In the first year you wrote **structured** (procedural) programs:

- **Methods** do the work: each one fulfills a single task and is steered by its parameters.
- **Data** is kept “on the side”: in variables, arrays (many values of the *same* type) and records (values of *different* types that belong together).

This works well for small programs. In larger ones, data and the methods working on it drift apart, and it gets hard to see who is allowed to change what.

Object-oriented programming (OO) puts data and logic back together: an **object** holds its data *and* the methods that work on that data. Each object is responsible for one part of the overall task.

1.1.1 Goals of OO

- **Reusability**: an object is a self-contained building block that can be used in many places.
 - **Single Responsibility Principle**: a class should have exactly one responsibility.
- **Encapsulation**: *how* an object works is hidden inside. Other code only uses its public interface (a “contract”), so the inside can change without breaking the outside.
- **Testability and extensibility**: a defined interface is easy to test, and an object can later get new capabilities.

1.1.2 What Is an Object?

An object has two sides that only make sense together:

- **State**: its data. A pupil has a first name, last name, date of birth and ZIP code.
- **Behavior**: its methods. A pupil can tell whether it is of full age, whether it is older than another pupil, or whether it lives nearby.

! Object = state + behavior

Behavior uses the state and may change it, and the state influences the result of the behavior. Only together do they make up an object.

The blueprint for objects is a **class**. It is similar to a **record**, but contains **not only** data, it **also** contains behavior.

1.1.3 Finding Classes

To design an object-oriented program, read the task description and look for the important **nouns**: they are candidates for classes. Then decide which data each class needs (state) and what it must be able to do (behavior).

Example: for a school administration, **Pupil** is an obvious class:

State (fields)	Behavior (methods)
FirstName, LastName	IsOfFullAge
DateOfBirth	IsOlderThan
ZipCode, City	LivesNearby

The next chapter turns this into C# code.

1.2 Classes and Objects

1.2.1 Classes Are Blueprints for Objects

A **class** describes what objects of a certain kind look like and what they can do. It is one level of structure *above* a method: where a method bundles a few statements into one task, a class bundles data and several methods into one unit.

A class defines three things:

- The **state** of its objects: which data every object carries. In C# these are **fields** and **properties**. The class only says *that* every pupil has a first name, not *which* name. The concrete values belong to the individual objects.
- The **behavior** of its objects: what every object can do. These are the **methods** of the class.
- What is visible from the outside (**public**) and what stays hidden inside (**private**). This hiding is called **encapsulation**: other code may only use the public parts, so the class can change its inner workings without breaking anyone.

As a rule of thumb, data is **private** and is reached from the outside through properties, while the methods that form the object's "interface" are **public**. Methods that only help other methods of the class do their work stay **private**.

1.2.1.1 Components of a Class

A typical class is made of these parts (you will meet each of them in the following chapters):

- A **unique name**, such as `Pupil`. By convention it is a noun starting with an uppercase letter.
- **private fields** that store the object's data.
- **public properties** that give controlled access to that data. In special cases, e.g. a computed value that is only used inside the class, a property can also be **private**.
- A **constructor** that sets up a new object.
- **public methods**: what other code can ask the object to do.
- **private helper methods** that split the work of the public methods into smaller steps.

1.2.1.2 Defining the Pupil Class

Each class usually lives in its own file with the same name. To create `Pupil.cs`, use *Add New Item* → *Class* in Visual Studio (or *Add* → *Class/Interface* in Rider):

```
public class Pupil
{
}
```

The class is still empty, but it is already a new data type: you could create `Pupil` objects now, they just wouldn't hold any data yet.

1.2.1.3 Properties in C#

The simplest way to give the class some data is an **automatic property**. Both IDEs have a shortcut for it: type `prop` and press TAB (twice in Visual Studio). The IDE inserts a template in which you only enter the data type and the name. Property names use *PascalCase*: every word starts with an uppercase letter, including the first one (`FirstName`, `DateOfBirth`).

```
public class Pupil
{
    public string FirstName { get; set; }
}
```

`{ get; set; }` means that the value can be read (`get`) and changed (`set`) from the outside. The chapter [Properties in Detail](#) explains what happens behind the scenes.

1.2.1.4 Class Diagram

A class with a few properties quickly becomes long in code. A **class diagram** shows the same structure at a glance, without any implementation details. Here is the class definition:

```
public class Pupil
{
    public string FirstName { get; set; }

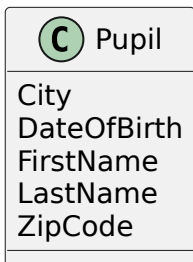
    public string LastName { get; set; }

    public int ZipCode { get; set; }

    public string City { get; set; }

    public DateTime DateOfBirth { get; set; }
}
```

And here is the matching class diagram. The box has the class name at the top, followed by its data:



The exercises specify their classes with such diagrams. The chapter [UML Class and Object Diagrams](#) explains the notation, e.g. the symbols for **public** and **private**.

1.2.1.5 Fields

A **field** is a variable that belongs to an object instead of to a method. Fields store the object's state. A pupil could store its data in these private fields:

```
public class Pupil
{
    private static int _nextId;
    private readonly DateOnly _dateOfBirth;
    private int? _zipCode;
}
```

Like a variable, each field has a data type and a name. There are a few important differences and rules, though:

- **Lifetime and scope:** a local variable only exists while its method runs. A field lives as long as its object and can be used by **all** methods of the class. It cannot be used from outside the class, and each object has its **own** values: changing `_zipCode` of one pupil does not affect any other pupil.
- **Always private:** other classes should not change an object's data directly. They go through properties or methods, which can check the new values.
- **Naming:** fields start with an underscore and a lowercase letter, e.g. `_dateOfBirth`. So you can tell at a glance whether a name refers to a field or to a local variable. The chapter [Naming Conventions](#) sums up all naming rules.
- **Initialization:** fields should get their starting value in the constructor, so that a new object never contains random or missing data.
- **readonly:** a [readonly](#) field can only be set in the constructor and never changes afterwards. A pupil's date of birth is a good example.
- **Nullable types:** `int?` means that the field may also hold **null**, i.e. "no value". `_zipCode` is **null** as long as no ZIP code is known.
- **static:** a field marked **static** does *not* belong to one object, but is shared by all objects of the class. The chapter [Static vs. Instance](#) explains why `_nextId` needs this.

1.2.2 Classes Are Templates

Think of a class as a stamp. The `Pupil` stamp has the fields `FirstName`, `LastName`, `DateOfBirth` and `ZipCode`, and the methods `IsOfAge`, `IsOlderThan` and `LivesNearby`. With this one stamp you can create as many pupil objects as you like. Each has its own, independent values (one pupil named `Max Muster` with ZIP 4060, another named `Erika Musterfrau` with ZIP 1010), and each can run the same methods on its own data.

1.2.3 Instantiation: Creating Objects Based on Classes

Creating an object from a class is called **instantiation**, and the object is called an **instance** of the class. The `Pupil` class defines the structure (`FirstName`, `LastName`, `DateOfBirth`, `ZipCode`, ...), and each instance fills it with its own values:

FirstName	LastName	DateOfBirth	ZipCode
Anna	Müller	2008-05-12	4060
Paul	Gruber	2006-07-30	(none)

The chapter [UML Class and Object Diagrams](#) shows how to draw such objects.

1.2.3.1 Creating and Using Objects

In code, objects are created with the `new` operator:

```
Pupil pupil = new Pupil();

pupil.FirstName = "Max";
pupil.LastName = "Muster";

Console.WriteLine($"{pupil.FirstName,-15} {pupil.LastName,-35}");
```

Here is what happens step by step:

1. `new Pupil()` reserves memory for a new object on the **heap**, the memory area for objects. A class is always a *reference type*.
2. `new` returns a **reference** to that object, a kind of address that says where it lives in memory. The variable `pupil` stores this reference, not the object itself.
3. With the **dot operator** (`pupil.FirstName`) you reach the properties and methods of exactly the object the variable refers to.

No object, no access

You can only use an object after it has been created. If a variable of type `Pupil` does not refer to any object yet (its value is `null`), `pupil.FirstName` fails at runtime with a `NullReferenceException`.

Since C# 9 you can omit the class name after `new` when the compiler already knows the type from the left side. This is called *target-typed new*:

```
Pupil pupil = new();
```

1.2.3.2 The Relationship Between Object and Class

Class and object are easy to mix up, so here is how they differ:

Class	Object
Exists exactly once in the program	There can be any number of them
Describes what objects look like (state, behavior)	Holds concrete values and executes the operations
Is fixed when you write the code	Is created (and removed) while the program runs
Blueprint for creating objects	Built from that blueprint

Each object is an *instance of* its class:

Class < instance of Object

1.2.3.3 Methods

So far the pupil only holds data. **Methods** add behavior: they describe what an object can *do*. A method of a class can read the object's data and can also change it, e.g. a method **Move()** of a horse could increase its position. In a class diagram, the methods are listed in a second section below the data:

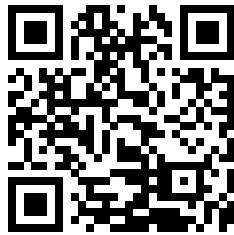


You already know how to write methods from the first year. The only difference is that these methods don't need the pupil's data as parameters: they belong to the object and can use its fields and properties directly.

Check your understanding of this chapter with the Classes and Objects Self-Check Quiz.

i Classes and Objects Self-Check Quiz

► [Take the quiz on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:

<https://app.novedu.at/ic2rwls9yp>

1.3 Constructors

1.3.1 Constructor Hooking into Instantiation

In the previous chapter, a pupil was created in two steps: first `new Pupil()`, then setting the properties one by one. That has a weakness: between these steps the object exists, but it is incomplete. Forget one assignment, and you have a pupil without a last name. A **constructor** solves this. It is special code that runs automatically *while* an object is being created, so the object is complete from its very first moment.

Syntax

A constructor looks like a method, with two differences:

- It is named **exactly like the class** (`Pupil`).
- It has **no return type**, not even `void`. It doesn't return anything, because `new` already delivers the new object.

A class can have several constructors with different parameter lists (*overloading*, see below).

If you don't write any constructor, C# creates a **default constructor** for you. It has no parameters and does nothing. That is why `new Pupil()` worked in the previous chapter.

Purpose

The constructor's job is to bring a new object into a **working state**, so that it can fulfill its purpose right away. To do this, it initializes the fields. It often takes parameters for the data an object cannot do without. A pupil without a name or a date of birth makes no sense, so the constructor demands them.

1.3.2 Instead of new and Set... Constructor

This constructor takes the pupil's core data and stores it:

```
/// <summary>
/// Supply the pupil's core data directly at instantiation time
/// </summary>
/// <param name="firstName"></param>
/// <param name="lastName"></param>
/// <param name="dateOfBirth"></param>
public Pupil(string firstName, string lastName, DateOnly dateOfBirth)
{
    FirstName = firstName;
    LastName = lastName;
    _dateOfBirth = dateOfBirth;
}
```

Now it's impossible to create a pupil without these three values: the compiler reports an error if an argument is missing. Note the naming: the parameters start with a lowercase letter (`firstName`), the properties with an uppercase letter (`FirstName`), and the field with an underscore (`_dateOfBirth`). So it's always clear which is which.

1.3.3 Overloading Constructors

Sometimes more data is known when an object is created. A second constructor with a longer parameter list can accept it. Having several constructors with the same name but different parameters is called **overloading**, just like with methods:

```
/// <summary>
/// The "full" constructor - every pupil needs a name and a date of birth
/// </summary>
public Pupil(string firstName, string lastName, DateOnly dateOfBirth)
{
    FirstName = firstName;
    LastName = lastName;
```

```

        _dateOfBirth = dateOfBirth;
    }

    /// <summary>
    /// Convenience overload for when the ZIP code is already known too
    /// Avoids code duplication ==> calls the constructor above
    /// </summary>
    public Pupil(string firstName, string lastName, DateOnly dateOfBirth, int zipCode) :
        ↪ this(firstName, lastName, dateOfBirth)
    {
        ZipCode = zipCode;
    }

```

The second constructor doesn't repeat the three assignments. With `: this(firstName, lastName, dateOfBirth)` it first calls the other constructor, which sets name and date of birth. Then its own body adds the ZIP code. This way the shared setup code exists only once. (This overload isn't part of the 01-Pupils requirements. It's only here to show the pattern.)

The compiler picks the constructor whose parameters match the arguments. *Target-typed new* works here too, as long as the type is clear from the left side:

```

// Pupil pupil = new(); // won't compile any more, now that we defined our own constructors!
Pupil pupil = new("Max", "Muster", new(2002, 02, 02));
Pupil pupil2 = new("Jane", "Doe", new(2003, 03, 03), 4060);

```

💡 Check yourself

Think about these questions before you read on:

- Why can't we use the *default* constructor any longer?
- Why don't we have to write `new Pupil(new DateOnly(...))`?
- What is the value of `pupil.ZipCode`, since that object was created *without* a ZIP code?
- `pupil` and `pupil2` are both *instances* of the same class. Are they *equal* (`==`)? Would they be equal if they had the same values?

❗ No more default constructor

As soon as you write *any* constructor yourself, C# no longer creates the parameterless default constructor. That's intended: if a pupil needs a name, `new Pupil()` should not work. If you do need a parameterless constructor as well, write one yourself with an empty parameter list.

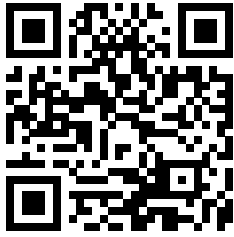
The meaning of this: inside a class, **this** is a **reference to the current instance**, i.e. the object whose method or constructor is running right now. `this._zipCode` means "the `_zipCode` of *this* pupil". Because it refers to one particular object, **this** is the opposite of **static** (see [Static vs. Instance](#)). The `: this(...)` call in a constructor's signature is a special use of the keyword. It will make more sense once we cover **base** (inheritance), so be patient for now.

Default values: every field has a value, even if the constructor doesn't set it. C# then fills in a default value. **Value types** get their type's default: `int` becomes 0, `double` becomes 0D, `bool` becomes `false`, `long` becomes 0L, `DateTime` becomes `DateTime.MinValue`, and so on. **Reference types** default to `null`. If the type is **not** declared as *nullable* (`string` instead of `string?`), the compiler warns you, and with `TreatWarningsAsErrors` that is a build error. Relying on default values is bad style: initialize every field explicitly in the constructor, so that it's obvious which value an object starts with.

Check your understanding of this chapter with the Constructors Self-Check Quiz.

i Constructors Self-Check Quiz

► [Take the quiz on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:

<https://app.novedu.at/qabe1fk12w>

1.4 Properties in Detail

From the outside, a property looks like a variable of the object: you read and assign it with the dot operator. On the inside, however, it consists of two small pieces of code, the **accessors**. **get** runs when the value is read, and **set** runs when a value is assigned. The **set** accessor receives the new value in a hidden parameter called **value**.

C# offers several ways to write a property, from a one-liner to a full block of code. This chapter shows them one after the other, and each time explains **which problem the longer form solves**. At the end, a table sums up which form to use when.

1.4.1 Automatic Properties

Most properties just store a value and hand it back. For them, the shortest form is enough, the **automatic property** (auto property). You only write the data type, the name, and which accessors exist:

```
public string FirstName { get; set; }
```

Using the property looks like using a variable:

```
Pupil pupil = new Pupil();
pupil.FirstName = "Max";
```

The value has to be stored somewhere. For an auto property, the compiler secretly creates a **private** field for it, the **backing field**, and fills in the accessor code. So the one-liner above behaves like this longer code:

```
private string _firstName; // created by the compiler, invisible to you

public string FirstName
{
    get { return _firstName; }
    set { _firstName = value; }
}
```

This is why a property is often called a public **wrapper around a field**: the field stores the value, and the property controls how it is read and written. Not every field needs a property, though. Data that is only used inside the class stays a plain **private** field.

1.4.1.1 Restricting Access

Often other classes may read a value but not change it. A pupil's first name, for example, is set in the constructor and should not be overwritten from anywhere else in the program. To achieve this, make only the **set** accessor **private**:

```
public string FirstName { get; private set; }
```

Now every class can read `pupil.FirstName`, but only code inside `Pupil` can assign it. An access modifier on an accessor can only **restrict** access further, never widen it.

If a value never changes at all once the object exists, leave out **set** completely. Such a **get-only** property can only be assigned in the constructor:

```
public DateOnly DateOfBirth { get; }
```

1.4.2 When an Auto Property Is Not Enough

An auto property stores every value it gets, without looking at it. As soon as the property has to **do something** when a value is assigned, the **set** accessor needs a body of its own. There are three typical reasons:

- **Validation:** reject values that make no sense. `Pupil.ZipCode` must stay between 1000 and 9999.
- **Transformation:** store the value in a cleaned-up form. A name typed as " Max " should be stored as "Max".
- **Reacting to a change:** do something extra whenever the value changes, e.g. write a message to a log.

Since C# 14 (.NET 10), the keyword **field** makes this easy. Inside an accessor, **field** stands for the hidden

backing field that the compiler creates. So you write a body only for the accessor that needs special code, and the other one stays automatic (`get;`).

Validation: `ZipCode` accepts only four-digit ZIP codes.

```
public int? ZipCode
{
    get;
    set
    {
        if (value is >= 1000 and <= 9999)
        {
            field = value;
        }
    }
}
```

`pupil.ZipCode = 4060;` runs the `set` accessor with `value = 4060`, which passes the check, so `field = value;` stores it. `pupil.ZipCode = 42;` runs the `set` accessor too, but the check fails, so the old value stays.

Transformation: `FirstName` removes the spaces around a name before storing it. With a single expression, the short `=>` form works (see [Expression Syntax](#)), and the setter can still be `private`:

```
public string FirstName
{
    get;
    private set => field = value.Trim();
}
```

Reacting to a change: `LastName` writes a message every time it changes, e.g. after a marriage:

```
public string LastName
{
    get;
    set
    {
        Console.WriteLine($"Changing last name from {field} to {value}");
        field = value;
    }
}
```

Note that the setter also runs when the constructor assigns `LastName = lastName;`. The first message is therefore “Changing last name from to Muster”, with an empty old value.

With `field`, you can start every property as a simple auto property `{ get; set; }`. If you later notice that some values must be rejected or cleaned up, you give the `set` accessor a body. You don’t need to add a field or rewrite `get`, and the code that uses the property doesn’t change at all.

1.4.3 Explicit Backing Fields

Before C# 14, the `field` keyword didn’t exist. To give an accessor a body, you had to declare the backing field yourself and write **both** accessors. This is called a **fully written-out** property. You will see it in almost all existing C# code, so you need to be able to read it:

```
private int? _zipCode;

public int? ZipCode
{
    get
    {
        return this._zipCode;
    }
    set
    {
        if (value is >= 1000 and <= 9999)
        {
            this._zipCode = value;
        }
    }
}
```

```
    }
}
```

It does exactly the same as the `ZipCode` with `field` above. A fully written-out property consists of these parts:

- A **private backing field** that holds the value. By convention its name starts with an underscore and a lowercase letter (`_zipCode`).
- The **property** itself, named like the field but starting with an uppercase letter (`ZipCode`). It usually has the same data type as the field.
- The **get accessor**, which returns the field's value.
- The **set accessor**, which receives the new value in `value` and writes it to the field, often after a check.

i Two correct ways

A property with an explicit backing field and a property with `field` do the same thing. When a class diagram lists the backing field (e.g. `-int? _zipCode`), write it out as shown in the diagram. Otherwise `field` is usually the shorter and safer choice: the compiler's hidden field has no name, so no other code in the class can write to it and bypass the check.

1.4.4 Expression Syntax

If an accessor does nothing but a single expression, you can shorten it with `=>` (*expression syntax*). You have already seen it in the `FirstName` setter above. It works for explicit backing fields as well:

```
private string _firstName;
public string FirstName
{
    get => this._firstName;
    set => this._firstName = value;
}
```

Expression syntax is only a shorter way of writing an accessor: `get => this._firstName;` means exactly the same as `get { return this._firstName; }`. As soon as an accessor needs more than one expression, like the `if` in `ZipCode`, you need the long form with braces.

1.4.5 Computed Properties

Some values don't need to be stored at all, because they follow from other data. A **computed property** has only a `get` accessor and no backing field. Its value is calculated again every time someone reads it. That's why it **cannot** have a `set` accessor: there is nowhere to store a value.

A pupil's full name is a good example. It follows from first and last name, so storing it separately would only risk that the three values contradict each other. Expression syntax makes this a one-liner:

```
public string FullName => $"{FirstName} {LastName}";
```

! Three different concepts

Don't confuse these forms. An *auto property* (`{ get; set; }`) stores a value in a hidden field. *Expression syntax* (`=>`) is just a shorter way to write an accessor. A *computed property* stores nothing and computes its value on every read, and it can be written in either the long or the short form. `public string FullName => ...;` (computed, no `set`) is something completely different from `public string FullName { get; set; }` (stored).

A pupil's age is computed too, because it changes over time, even if nothing in the object changes. It depends on the date of birth and today's date:

```
/// <summary>
/// How many whole years lie between the date of birth and today
/// </summary>
public int Age
{
    get
    {
        int years = DateTime.Now.Year - DateOfBirth.Year;
    }
}
```

```

DateTime fullYearsDate = DateOfBirth.AddYears(years);
if (fullYearsDate > DateTime.Now)
{
    // This year's birthday hasn't happened yet
    years--;
}
return years;
}

```

1.4.5.1 Complexity of Computed Properties

Whoever uses a property expects it to behave like a variable: reading it should be fast and should not change anything. A computed property must live up to that expectation. So keep it **simple**:

- **Fast:** no expensive work, such as big loops or creating many objects. Someone might read the property a thousand times in a loop without suspecting that each read is expensive.
- **Short:** a few lines at most, ideally just one.
- **No complex business logic,** and no side effects. Reading a value must not change the object.
- Methods called from the property **count too:** a one-line property that calls an expensive method is still expensive.

If you need more logic than that, write a **method** instead. A method call tells the reader “work is being done here”, and its result can be stored in a variable if it’s needed more than once.

1.4.6 Handling Invalid Values

What should a setter do when **value** fails the check? There are three options, each with an advantage and a drawback:

1. **Don’t set it at all:** the object never gets an incorrect value, but the caller doesn’t learn that something went wrong.
2. **Throw an exception:** the object never gets an incorrect value and the caller is notified, but the program stops unless the caller handles the exception. You will learn about exceptions later in the year.
3. **Set a meaningful default value instead:** the object never gets an incorrect value and the program keeps working, but the caller gets no feedback about the invalid value.

Which option fits best is up to the author of the class and depends on the specific value and on what the program is for. `Pupil.ZipCode` uses option 1: an invalid ZIP code is simply ignored, as the exercise requires.

💡 Use the properties inside the class, too

The class itself should use its properties as well, instead of accessing a backing field directly, even if that means working with a **private** setter. Otherwise a constructor could write `_zipCode = 42;` and bypass the check in the setter. When all code goes through the property, the checks always run and are written in one place only. With **field** this happens automatically, because there is no named backing field to write to.

1.4.7 Which Property Syntax When?

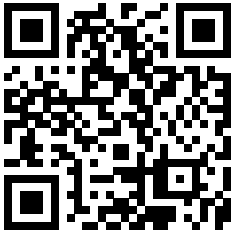
You need ...	Use	Example
a value that can be read and changed from anywhere	auto property	<code>public string City { get; set; }</code>
a value others may read, but only the class may change	auto property with private set	<code>public string FirstName { get; private set; }</code>
a value that never changes after the constructor	get-only auto property	<code>public DateOnly DateOfBirth { get; }</code>
to check, clean up or react to a new value	set with a body, using field	<code>ZipCode, FirstName, LastName</code> above
the same, when the backing field must be visible (e.g. listed in a class diagram)	explicit backing field	<code>private int? _zipCode; + ZipCode</code>

You need ...	Use	Example
a value that follows from other data	computed property	<code>public string FullName => ...;</code>
expensive or complex logic	a method, not a property	<code>public bool IsOlderThan(Pupil other)</code>

Check your understanding of this chapter with the Properties Self-Check Quiz.

Properties Self-Check Quiz

► [Take the quiz on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:

<https://app.novedu.at/6h5wa7oht5>

1.5 Static vs. Instance

So far, everything in a class belonged to the individual objects: every pupil has its **own** first name, its own date of birth, its own ZIP code. Members like these are called **instance members**, and they remain the **normal case**.

Sometimes, however, a value belongs to the class as a whole rather than to one object. The 01-Pupils exercise has such a case: every pupil gets a unique, ever-increasing Id. To hand out the next id, somebody has to remember which id comes next. That counter can't be stored in each pupil, because every pupil would have its own copy and they would all start at the same number. It must exist **once for the whole class**. Members like that are marked with the keyword **static**.

1.5.1 Fields

A **static** field exists exactly once, no matter how many objects there are, and **all instances share it**. When one pupil changes it, all other pupils see the new value:

```
public class Pupil
{
    private static int _nextId = 1;

    public int Id { get; }

    public Pupil(string firstName, string lastName, DateOnly dateOfBirth)
    {
        Id = _nextId;
        _nextId++;
        // ... remaining initialization
    }
}
```

Every time a new Pupil is created, the constructor copies the shared `_nextId` into the pupil's own `Id`, then increases the counter for the *next* pupil. The first pupil gets id 1, the second id 2, and so on. That's why `_nextId` must be **static**, while `Id` is a normal instance property: every pupil has its own id, but they all share one counter.

Side effects

Use **static** fields **very carefully**. A change made through one object silently affects all the others. This is called a *side effect*, and it's a common source of bugs that are hard to find. It can also make tests depend on each other, because the shared value survives from one test to the next.

1.5.2 Methods

Methods can be **static**, too. A **static** method belongs to the class, not to an object, so it runs **without** any instance. That's why you call it via the **class name**: `Console.WriteLine(...)` and `Math.Max(...)` are static methods, and so were all the methods you wrote in the first year.

This has a few consequences:

- A **static** method **cannot** use instance fields or properties, and **cannot** use `this`: there is no “current object” whose data it could access. It *can* use **static** fields.
- Its only way to work with objects is to receive them as **parameters**.
- Apart from that it's a normal method: it can be overloaded, and access modifiers like **public** and **private** apply as usual.

Although static methods belong to the class, good static methods are the **opposite** of side-effect-prone code. They work **only** with their parameters and return a result, without keeping any data between calls. We call such methods **stateless**. They are easy to understand and easy to test, which is why they were all you needed in the first year. If a method doesn't use any instance data, the IDE often suggests making it **static**.

1.5.3 Static Class

If a class contains **only** static members, you can declare the class itself as **static class**. This is typical for a collection of helper methods that don't need any state. The best-known example is `System.Math`: `Math.Sqrt`, `Math.Round` and `Math.Max` are all static, and a “math object” would make no sense.

Note

You **cannot** create an instance of a **static class**. `new Math()` does not compile.

1.5.4 `const`, `readonly` and `static` Compared

Three keywords change how a field behaves, and they are easy to mix up: `const`, `readonly` and `static`. They answer two different questions:

- **How many copies exist?** One per object (instance field), or one for the whole class (**static**, and every **const**).
- **When can the value change?** Never, because it is fixed in the code (**const**); once, in the constructor (**readonly**); or any time (neither).

This `HorseRace` class uses all of them:

```
public class HorseRace
{
    public const int MaxSteps = 20;
    public static readonly DateTime StartTime = DateTime.Now;
    private static int _raceCount;
    private readonly Horse[] _horses;

    public HorseRace(Horse[] horses)
    {
        _horses = horses;
        _raceCount++;
    }
}
```

const: a constant, fixed when you write the code. `MaxSteps` is 20 in every race and in every run of the program. The compiler writes the value directly into every place where `MaxSteps` is used, so it must be known at compile time: a number, a `bool`, a `char` or a string literal. `DateTime.Now` or `new Horse[5]` are not possible. A constant belongs to the class, not to an object, so you use it via the class name: `HorseRace.MaxSteps`. It is automatically static, and writing **static const** is even a compile error.

readonly: set once, then fixed. `_horses` gets its value in the constructor, and after that nobody can assign a different array to it, not even the class itself (`_horses = ...`; in a method is a compile error). Unlike a constant, the value is only known at runtime: each race gets the horses that are passed to its constructor. So every object has its **own** **readonly** value.

static: one copy for the whole class. `_raceCount` exists once and counts all races ever created. Without **readonly**, it can change at any time, with the side effects described [above](#).

static readonly: one copy, set once at runtime. `StartTime` exists once for the whole class and never changes, but its value is only known when the program runs, so it can't be a **const**. Use **static readonly** for constants that the compiler can't compute: a `DateTime`, an array, an object.

Keyword	Copies	Value is set ...	Can change later?	Example
<i>(none)</i>	one per object	anywhere in the class	yes	<code>private int _position;</code>
readonly	one per object	at the declaration or in the constructor	no	<code>private readonly Horse[] _horses;</code>
static	one for the class	anywhere in the class	yes	<code>private static int _raceCount;</code>
static readonly	one for the class	at the declaration	no	<code>public static readonly DateTime StartTime</code>

Keyword	Copies	Value is set ...	Can change later?	Example
<code>const</code>	one for the class	in the code, at compile time	no	<code>public const int MaxSteps = 20;</code>

To choose, ask the two questions from above:

- Is the value the same for every object and known when you write the code? Use **const**.
- Is it the same for every object, but only known at runtime? Use **static readonly**.
- Does each object get its own value that never changes after the constructor? Use **readonly**.
- Does the value change while the program runs? Leave **readonly** out, and add **static** only if all objects must share it.

⚠️ readonly protects the reference, not the object

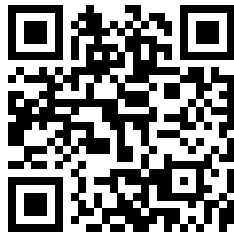
For an array or another object, **readonly** only means that the field always refers to the **same** array. The contents can still change: `_horses[0] = new Horse("Lara");` compiles and replaces the first horse, and sorting the array rearranges its elements. **readonly** prevents only `_horses = new Horse[3];`.

For properties, the same idea is called a **get-only** property (`public int Age { get; }`, see [Properties in Detail](#)): like a **readonly** field, it can only be assigned in the constructor.

Check your understanding of this chapter with the Static vs. Instance Self-Check Quiz.

i Static vs. Instance Self-Check Quiz

▶ [Take the quiz on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:
<https://app.novedu.at/ajlmgy4tp5>

1.6 Enumerations (enum)

You have already used an `enum` now and then, e.g. `ConsoleColor.Red` or `ConsoleKey.Enter`. Now it's time to understand what an `enum` actually is.

Often a value can only take a few fixed options: a traffic light is red, yellow or green, and a pupil can be checked against voting age, full age or retirement age. You could encode such options as numbers (0 = voting age, 1 = full age, ...) or as strings. However, numbers mean nothing to the reader, and a typo in a string is only noticed when the program runs. An **enumeration** gives each option a **name** that the compiler knows and checks.

An `enum` is a data type that consists of a fixed **list of named constants**. The 01-Pupils exercise needs one for `AgeType`:

```
enum AgeType
{
    VotingAge, // 0
    FullAge,   // 1
    RetirementAge // 2
}
```

A variable of type `AgeType` can only hold one of these three values. If you write `AgeType.FulAge`, the compiler reports the typo immediately.

Behind the scenes, every entry is a **unique number** of an integer type. By default it's `int`, and that's fine: `byte`, `short` and `long` are also possible, but they only matter in special cases such as talking to hardware or code written in other languages. Unless you say otherwise, the entries are numbered 0, 1, 2, ... in the order they appear in the code, as the comments above show.

This is how you work with an `enum`:

```
AgeType type = AgeType.FullAge;
Console.WriteLine(type); // Output: FullAge

type = (AgeType)2;
Console.WriteLine(type); // Output: RetirementAge

int value = (int)AgeType.VotingAge;
Console.WriteLine(value); // Output: 0

type = Enum.Parse<AgeType>("FullAge"); // TryParse would be the smarter choice
Console.WriteLine(type); // Output: FullAge
```

Line by line:

- An entry is always written with the type name in front: `AgeType.FullAge`. `Console.WriteLine` prints its **name**, not its number.
- A cast converts in both directions: `(AgeType)2` turns the number 2 into `RetirementAge`, and `(int)AgeType.VotingAge` gives the number 0.
- `Enum.Parse<AgeType>("FullAge")` turns a string, e.g. read from a file, into the matching entry. It throws an exception if the string doesn't match any entry. That's why `Enum.TryParse` is the better choice, following the same `TryParse` pattern you know from `int.TryParse`.

You can also assign the numbers yourself, e.g. `FullAge = 18`. Usually the default numbering is fine, and it's all `AgeType` needs here.

Check your understanding of this chapter with the Enumerations Self-Check Quiz.

i Enumerations Self-Check Quiz

► [Take the quiz on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:

<https://app.novedu.at/vgrypzov5v>

1.7 Naming Conventions

The compiler doesn't care whether a property is called `FirstName`, `firstname` or `fn`. People do. Code is read far more often than it is written, and names are what a reader sees first. C# developers all over the world follow the same **naming conventions**, based on [Microsoft's guidelines](#). If you follow them too, anyone can tell at a glance whether a name is a class, a property, a field or a local variable, without scrolling to its declaration.

The conventions answer two questions: **how** a name is written (upper- or lowercase, underscore) and **what** a name should say.

1.7.1 How Names Are Written

C# uses two styles of capitalization:

- **PascalCase**: every word starts with an uppercase letter, including the first: `Pupil`, `FirstName`, `IsOlderThan`, `HorseRace`.
- **camelCase**: like PascalCase, but the first letter is lowercase: `firstName`, `dateOfBirth`, `startNumber`.

Neither style uses spaces or underscores between words. The only underscore is the prefix of private fields.

The rule of thumb: everything that is **visible from outside** a class (the class itself, its public properties, methods and constants) is written in **PascalCase**. Everything that is **local or private** data (parameters, local variables, private fields) starts with a **lowercase letter**, and private fields additionally get a leading **underscore**. The one exception: constants are always PascalCase, even when they are **private** (`DelayMilliseconds` in `HorseRace`).

Element	Style	Example
Class, enum, record	PascalCase	<code>Pupil</code> , <code>HorseRace</code> , <code>AgeType</code>
Entry of an enum	PascalCase	<code>AgeType.FullAge</code>
Property	PascalCase	<code>FirstName</code> , <code>ZipCode</code> , <code>IsFinished</code>
Method	PascalCase	<code>IsOlderThan</code> , <code>PrintResults</code> , <code>Move</code>
Constant (<code>const</code>)	PascalCase	<code>MaxSteps</code> , <code>DelayMilliseconds</code>
private field (also <code>static</code>)	<code>_</code> + camelCase	<code>_zipCode</code> , <code>_horses</code> , <code>_nextId</code>
Parameter	camelCase	<code>firstName</code> , <code>dateOfBirth</code>
Local variable	camelCase	<code>years</code> , <code>fullYearsDate</code>
Interface (later this year)	<code>I</code> + PascalCase	<code>IComparable</code>

The different styles also help where names are similar: a parameter `name`, a field `_name` and a property `Name` can exist side by side in one class, and you always know which one is meant.

! Your IDE checks this

The exercises ship an `.editorconfig` file with exactly these rules. Rider and Visual Studio underline names that break them and suggest a fix. The build still succeeds, so it's up to you: fix these hints right away instead of ignoring them.

1.7.2 What Names Should Say

A correctly capitalized name can still be a bad name. These guidelines help you choose good ones:

- **Classes are nouns**, in the singular: `Pupil`, `Horse`, `HorseRace`. An object of the class *is* one such thing. A class name like `Pupils` or `ManagePupils` is a hint that something is wrong.
- **Methods are verbs**, because they *do* something: `Move()`, `PrintResults()`, `AssignRanks()`. A method that answers a yes/no question starts with `Is`, `Has` or `Can` and returns `bool`: `IsOlderThan(other)`, `IsOfAge(type)`.
- **Properties are nouns or adjectives**, because they describe the object: `FirstName`, `Age`, `Position`. A `bool` property also starts with `Is`, `Has` or `Can`: `IsFinished`, `HasZipCode`.
- **Collections are plural**: an array of horses is called `_horses`, not `_horse` or `_horseArray`.

- **Say what it is, not how it is stored:** `DateOfBirth`, not `DateString` or `Dob`.
- **No abbreviations** unless everybody knows them (`Id`, `Html`). Autocompletion writes long names for you, so `startNumber` costs nothing compared to `sn`.
- **English only:** `C#` keywords and the .NET class library are English, so `Pupil.IsOfAge` reads naturally, while a mix like `Schueler.IsVolljaehrig` does not.
- **One class per file**, and the file has the same name as the class: `Pupil` lives in `Pupil.cs`.

💡 Renaming is cheap

If you notice that a name no longer fits, rename it with your IDE's rename refactoring (*Rename* in the context menu; `Ctrl+R`, `R` in Visual Studio, `Ctrl+R`, `R` or `Shift+F6` in Rider, depending on its keymap). It updates every usage in the whole solution, so there is no excuse for keeping a misleading name.

Check your understanding of this chapter with the Naming Conventions Self-Check Quiz.

i Naming Conventions Self-Check Quiz

▶ [Take the quiz on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:
<https://app.novedu.at/6cjn2881cq>

1.8 UML Class and Object Diagrams

UML (Unified Modeling Language) is a standardized set of diagram types for describing software design independently of any programming language. The **class diagram** is UML’s diagram for *static structure*: it shows the classes in a system, their members (fields, properties, methods), and how classes relate to one another — without any of the runtime behavior or control flow.

PlantUML is a text-based way to author UML diagrams: you write the diagram as plain text (`@startuml ... @enduml`), and a renderer turns it into an image. See the [class diagram](#) and [object diagram](#) syntax references for the full notation.

This chapter covers two UML diagram types: **class diagrams** show the *blueprint* of the classes, **object diagrams** show the objects that exist at one moment while the program runs.

1.8.1 Class Diagrams

i Why class diagrams matter

Class diagrams matter in software development because they let you communicate and review a design *before* (or without) reading all the code: given a class’s public members and its relationships to other classes, another developer can understand what it’s responsible for and how to use it.

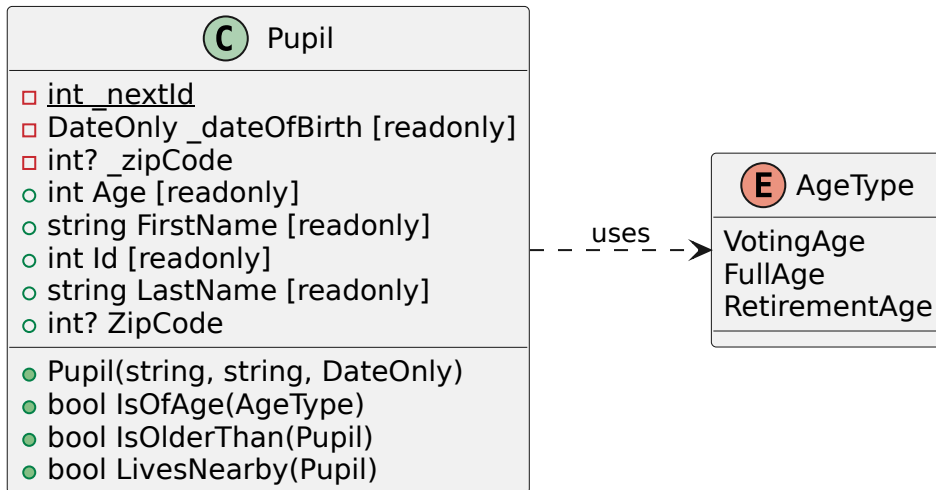
1.8.1.1 Class Diagram Notation

Symbol	Meaning	Rendered As
<code>+</code>	public member	green circle icon before the member
<code>-</code>	private member	red square icon before the member
<code>{static}</code> (prefix)	static member	member name shown underlined
<code>type name</code>	field or property	plain attribute line, e.g. <code>- int age</code>
<code>returnType</code>	method	plain line with the visibility icon, e.g. <code>+ bool</code>
<code>name(params)</code>		<code>isOfAge(AgeType)</code>
<code>enum Name { ... }</code>	an enum	its own box, marked with an enum icon/ <code><<Enumeration>></code> label, listing values
<code>..></code>	dependency (“uses”)	dashed line with an open (unfilled) arrowhead

PlantUML has no built-in symbol for **readonly** or a **private** setter, so this repo’s diagrams add a bracketed suffix as a plain-text convention — PlantUML just prints whatever text follows a member, so this isn’t parsed as anything special, just displayed:

Suffix	Meaning	Rendered As
<code>[const]</code>	compile-time constant	literal trailing text — no icon or styling of its own
<code>[readonly]</code>	no public setter at all	literal trailing text — no icon or styling of its own
<code>[private set]</code>	public getter, but the setter is private	literal trailing text — no icon or styling of its own

1.8.1.2 Illustration: the Pupil Class



Same diagram, as plain text (for reference — this block isn't rendered):

```
@startuml
left to right direction

enum AgeType {
    VotingAge
    FullAge
    RetirementAge
}

class Pupil {
    {static} -int _nextId
    -DateOnly _dateOfBirth [readonly]
    -int? _zipCode
    +Pupil(string, string, DateOnly)
    +int Age [readonly]
    +string FirstName [readonly]
    +int Id [readonly]
    +string LastName [readonly]
    +int? ZipCode
    +bool IsOfAge(AgeType)
    +bool IsOlderThan(Pupil)
    +bool LivesNearby(Pupil)
}

Pupil ..> AgeType : uses

@enduml
```

1.8.2 Object Diagrams

A class diagram shows the *blueprint*: which data and methods every object of a class has. An **object diagram** shows what exists at one particular **moment** while the program runs: the concrete objects, the values of their data, and which objects refer to which. You can think of it as a snapshot of the heap.

i Why object diagrams matter

Object diagrams matter because many bugs are not about what a class looks like, but about which objects exist at runtime and how they are connected: drawing the objects step by step lets you follow a program the way you did desk checks in the first year, and see mistakes such as a missing object or a reference to the wrong one.

An object diagram differs from a class diagram in a few ways:

- **Concrete values instead of types:** a class diagram says `+string FirstName`, an object diagram says `FirstName = "Anna"`. All data of the object is shown, including **private** fields, because it all belongs to the object's state.

- **No methods:** all objects of a class have the same methods, so they would add nothing to the snapshot.
- **No static fields:** they belong to the class, not to an object. `Pupil._nextId` therefore never appears in an object box.
- **Many boxes per class:** there is one box per *object*, so a class can appear many times, or not at all if no object of it exists at that moment.

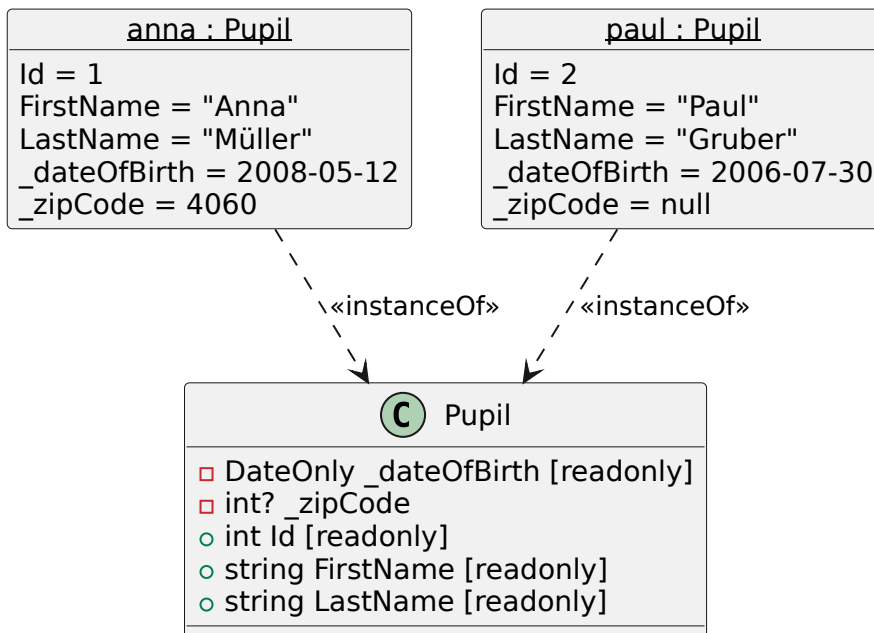
1.8.2.1 Object Diagram Notation

Symbol	Meaning	Rendered As
object "name : Class" as alias __text__	an object named name of class Class underlined text	a box with name : Class as heading UML underlines object headings to tell them apart from classes
" : Class" (no name)	an anonymous object	heading shows only : Class , e.g. an array element
name = value	current value of a field or property	plain line in the box, e.g. <code>ZipCode = 4060</code>
a --> b : label	link: a holds a reference to b	solid arrow, labeled with the field that holds the reference
allowmixing	classes and objects in the same diagram	lets class boxes appear next to object boxes
a ..> Class : <<instanceOf>>	a is an instance of Class	dashed arrow labeled «instanceOf»

The `alias` is only used inside the PlantUML text to draw links (`anna, race`). It is not shown in the diagram.

1.8.2.2 Illustration: Two Pupil Objects

The two pupils from the chapter [Classes and Objects](#) as an object diagram. Each object has its own values, and both are instances of the same class. `paul` was created without a ZIP code, so his `_zipCode` is `null`.



Same diagram, as plain text (for reference — this block isn't rendered):

```

@startuml
allowmixing

class Pupil {

```

```

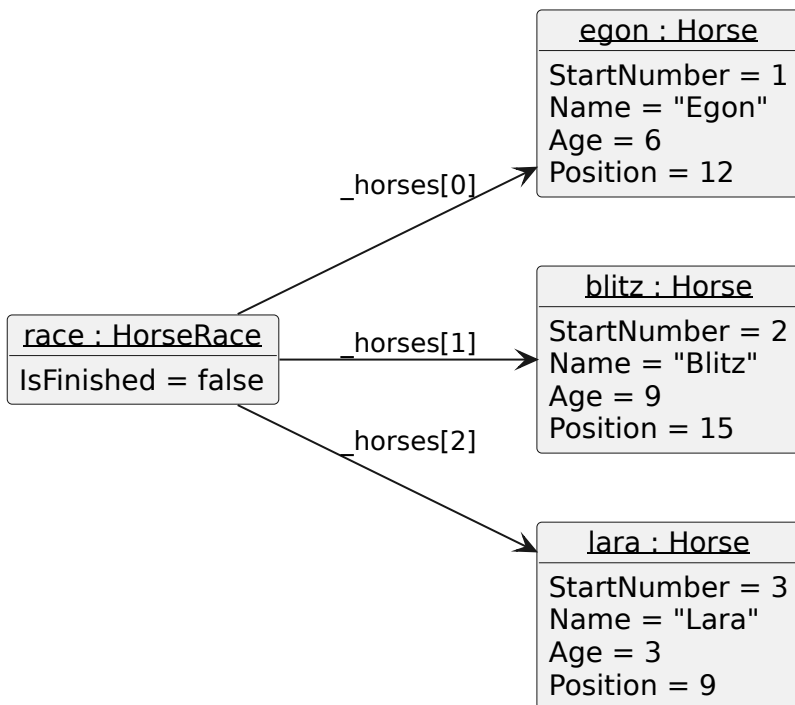
-DateOnly _dateOfBirth [readonly]
-int? _zipCode
+int Id [readonly]
+string FirstName [readonly]
+string LastName [readonly]
}
object "__anna : Pupil__" as anna {
  Id = 1
  FirstName = "Anna"
  LastName = "Müller"
  _dateOfBirth = 2008-05-12
  _zipCode = 4060
}
object "__paul : Pupil__" as paul {
  Id = 2
  FirstName = "Paul"
  LastName = "Gruber"
  _dateOfBirth = 2006-07-30
  _zipCode = null
}
anna ..> Pupil : <<instanceOf>>
paul ..> Pupil : <<instanceOf>>

@enduml

```

1.8.2.3 Illustration: Objects That Refer to Each Other

Object diagrams really pay off when objects hold references to other objects. In the 03-HorseRace exercise, a `HorseRace` object keeps its horses in the array `_horses`. This snapshot shows a race with three horses in the middle of a run:



Each arrow is one element of the array: `_horses[0]` holds a reference to `egon`, and so on. The horses themselves don't know the race, so there are no arrows back. After the next `MoveHorses()` call, the `Position` values change, but the arrows stay the same. A second snapshot would show exactly that.

Check your understanding of this chapter with the UML Class and Object Diagrams Self-Check Quiz.

i UML Class and Object Diagrams Self-Check Quiz

► [Take the quiz on app.novedu.at](https://app.novedu.at)



Scan the code, or type this address into your browser:

<https://app.novedu.at/x7zofg974i>

1.9 Glossary

Term	Description	Syntax
Class	Data type, blueprint, template	<code>public class Pupil {}</code>
Object	Instance, object instance, concrete manifestation of a class	<code>Pupil p = new Pupil();</code>
Object diagram	UML diagram that shows the objects existing at one moment at runtime, with their current values and the references between them.	<code>object "__anna : Pupil__" as anna { ... }</code> (PlantUML)
Object reference	Variable of the class's type that points to a concrete object instance (or <code>null</code>). The variable lives on the stack and holds the memory address of the object on the heap (or <code>null</code>).	<code>Pupil p;</code> <code>p = new Pupil();</code> <code>p = null;</code>
Constructor	Method that is called with <code>new</code> while an object is being instantiated (created).	Method with no return value, method name == class name
Default constructor	A parameterless constructor exists automatically, unless you define a custom constructor.	<code>public Pupil(){} </code>
Custom constructor	Used to shape/initialize the new object with specific parameters. This is how, e.g., mandatory properties of an object are established at the very start of its lifetime. Example: <code>Pupil p = new Pupil("Susi", "Huber");</code>	<code>public Pupil(string firstName, string LastName) {}</code>
Properties	Data that (can) distinguish one object from another. Aka properties, member variables, instance variables	<code>public string FirstName {get; set;}</code>
Behavior	Instance methods (object methods) are methods that run on an object. They represent the object's abilities. They can also change the object's properties.	
Publicly visible	A class/constructor/property/method is public and can be used from any other project in the same VS solution - see unit tests.	<code>public</code>
Private	Private data or methods that only the class itself needs to fulfill its tasks.	<code>private</code>
Read-only properties	Only read access to an object's properties is publicly allowed. Setting or changing the property may only happen from inside the class itself.	<code>public string FirstName {get; private set;}</code>
field keyword	Since C# 14: inside a property accessor, <code>field</code> is the backing field the compiler creates. Lets you add a check to an auto property without declaring the field yourself.	<code>public int? ZipCode { get; set { if (...) field = value; } }</code>
Computed properties	Only a get accessor is defined for the property. The property's value can't be set because it is freshly computed on every access.	<code>public int Age {get { ... } }</code>
Class methods	Can only be invoked on the class name, not on object instances. Examples: <code>Console.WriteLine(),</code> <code>CsvImporter.ReadCsvFile()</code>	<code>static</code> methods of a class

Term	Description	Syntax
Static field	Exists once for the whole class and is shared by all objects.	<code>private static int _nextId;</code>
Constant	Value fixed in the code at compile time; belongs to the class and is used via the class name.	<code>public const int MaxSteps = 20;</code>
Read-only field	Set at the declaration or in the constructor, never changed afterwards. For arrays and objects only the reference is fixed, not the contents.	<code>private readonly Horse[] _horses;</code>